

Connected Vehicle Intelligence Hackathon

Solution Document Template

Submission Format: PDF (export of this document)

To be Submitted by: [Team Name]

Team Members & Roles: [Name – role – email, for each member]

Problem Space Chosen: [e.g. predictive maintenance, EV charging, or your own]

Repository URL: [Link]

Demo Video URL (≤ 5 min): [Link]

Date of Submission: [DD/MM/YYYY]

Table of Contents

- [1. Executive Summary](#)
- [2. Problem Statement & Validation](#)
 - [2.1 Problem Statement](#)
 - [2.2 Evidence & Validation](#)
 - [2.3 Impact & Success Metrics](#)
- [3. Solution Description](#)
 - [3.1 Solution Overview & User Journey](#)
 - [3.2 Key Value Proposition](#)
 - [3.3 Innovative Ideas](#)
- [4. Feature List](#)
- [5. Solution Architecture \(High-Level Design\)](#)
 - [5.1 Architecture Overview](#)
 - [5.2 Technology Stack & Justification](#)
 - [5.3 Data Architecture](#)
 - [5.4 Deployment View](#)
- [6. Low-Level Design](#)
 - [6.1 Layering & Separation of Concerns](#)
 - [6.2 Design Principles Applied](#)
 - [6.3 Design Patterns Used](#)
 - [6.4 Interfaces, Contracts & Runtime Flows](#)
 - [6.5 Algorithms & Data Structures](#)
- [7. Non-Functional Requirements & Performance Benchmarks](#)
- [8. Security & Compliance](#)
- [9. Test Strategy](#)
- [10. Observability](#)
- [11. AI / ML Component \(if used\)](#)
- [12. Architecture Decisions, Risks & Future Enhancements](#)
- [13. Demo Video \(5 Minutes Maximum\)](#)
- [14. Repository Checklist](#)
- [15. Conclusion](#)
- [16. Declarations](#)
- [17. Appendix \(if any\)](#)

1. Executive Summary

Brief overview (half a page maximum) of:

- The problem you chose and who it affects
- What your solution does, in one or two sentences
- Key results achieved (throughput, latency, impact numbers)
- Unique features, innovations or optimisations you implemented

[Write your answer here]

2. Problem Statement & Validation

2.1 Problem Statement

- **Format:** “[User] needs a way to [job to be done] because [pain], which today costs [measurable impact].”
- **Primary User:** Fleet manager, dealer, lender, driver or OEM — name one
- **Secondary Stakeholders:** Others affected by the problem

[Write your answer here]

2.2 Evidence & Validation

- **Facts vs Assumptions:** Separate what you know from what you assume
- **Validation Method:** Published study, public dataset, simulation, or interviews
- **Existing Alternatives:** Aftermarket dongles, OEM portals, Motorq Fuse, other telematics tools — and why they fall short

Evidence / Assumption	Source or Method	What It Shows	Confidence
<i>[e.g. Idling wastes fuel in large fleets]</i>	<i>[Study / dataset / interviews]</i>	<i>[Size of pain in time, money or risk]</i>	<i>[High / Medium / Low]</i>

2.3 Impact & Success Metrics

Metric	Baseline Today	Target	How Measured / Estimated
<i>[e.g. Unplanned breakdowns per 1,000 vehicles / month]</i>			<i>[Simulation, back-test, benchmark]</i>

- **Scale of Impact:** Who benefits, by how much, at 10K vs 100K vehicles

- **Wider Impact:** Safety, cost, environment or compliance, where relevant

[Write your answer here]

3. Solution Description

3.1 Solution Overview & User Journey

- **What It Does:** Plain-language description of the solution
- **User Journey:** Step by step, e.g. vehicle event → detection → alert → action → outcome
- **Screenshot:** At least one screenshot of the working product

[Write your answer here]

3.2 Key Value Proposition

- **Customer Job:** What the user is trying to get done
- **Pain Relieved:** What gets easier, cheaper or safer
- **Gain Created:** New capability the user did not have
- **Differentiation:** Why your solution, not the alternatives

[Write your answer here]

3.3 Innovative Ideas

- **Up to Three Ideas:** A technique, data insight, user experience or business model
- **What Is New:** Compared against prior work you studied
- **Evidence:** Proof that each idea works

[Write your answer here]

4. Feature List

Every feature must be traceable to code. Priority uses MoSCoW (Must / Should / Could / Won't). Status: Done / Partial / Planned. Give the demo video timestamp for Done features.

ID	Feature	User Story	Priority	Status	Code Path	Video
F-01	<i>[Real-time harsh-braking alert]</i>	<i>[As a fleet manager I want ... so that ...]</i>	<i>[Must]</i>	<i>[Done]</i>	<i>[services/alerts/]</i>	<i>[01:42]</i>
F-02						
F-03						

5. Solution Architecture (High-Level Design)

5.1 Architecture Overview

- **System Context Diagram:** C4 level 1 — your system, users and external systems (OEM clouds, identity provider, LLM API, data warehouse)
- **Container Diagram:** C4 level 2 — every service, data store and broker, with the protocol on each arrow (MQTT, HTTPS, gRPC, Kafka)
- **Data Flow:** Path of one telemetry event from vehicle to insight, with expected latency per hop

[Insert diagrams and explanation]

5.2 Technology Stack & Justification

Layer	Choice	Why This, and What You Rejected
Ingestion / Messaging		<i>[e.g. Kafka over RabbitMQ for replay and partitioned ordering]</i>
Stream / Batch Processing		
Relational / NoSQL / Cache / Search / Vector		
Backend / Frontend		
ML / AI		
Infrastructure / CI-CD / Observability		

5.3 Data Architecture

- **ER Diagram:** Relational core normalised to 3NF; list any deliberate denormalisation and why
- **Polyglot Map:** Which data lives in which store, with its CAP choice (CP or AP)
- **Capacity Estimate:** Events/sec, bytes/event, daily and yearly volume, partition keys, hot / warm / cold retention
- **Query Optimisation:** Top three queries with EXPLAIN ANALYZE before and after

Query	Before (ms)	After (ms)	Change Made

5.4 Deployment View

- **Deployment Diagram:** Clusters, namespaces, replicas, autoscaling, networking, secrets
- **Cloud-Agnostic Approach:** How the same deployment runs on a second cloud or locally

[Write your answer here]

6. Low-Level Design

6.1 Layering & Separation of Concerns

- **Architecture Style:** Layered, hexagonal (ports and adapters) or clean architecture — justify
- **Layer Diagram:** Internal layers of at least one core service and the allowed dependency direction
- **Folder Structure:** Repository tree, two levels deep

Layer	Responsibility	Must Not
Presentation / API	HTTP, WebSocket, validation, auth checks, DTO mapping	Contain business rules or SQL
Application / Service	Use cases, transactions, orchestration	Depend on a specific database or broker
Domain	Entities, value objects, business rules	Import framework or infrastructure code
Infrastructure	Repositories, broker clients, external APIs, caches	Leak vendor types into the domain

6.2 Design Principles Applied

- **SOLID:** [Where applied (file / class) and what it gave you]
- **12-Factor App:** [Config in environment, stateless processes, disposability]
- **Idempotency, Fail-Fast, Least Privilege, DRY, KISS:** [Where applied]

6.3 Design Patterns Used

Pattern	Problem It Solves in Your System	Location in Code
[Adapter]	[Maps each OEM's payload format to one canonical event]	
[Circuit Breaker / Retry]	[Protects the API when a downstream store is slow]	
[Outbox / CQRS / Saga / Repository / Strategy / Observer]		

6.4 Interfaces, Contracts & Runtime Flows

- **API Contract:** OpenAPI / AsyncAPI link; versioning, pagination, error format, rate limits
- **Event Schemas:** Topics, keys, schema format (Avro / Protobuf / JSON Schema), schema evolution

- **Sequence Diagrams:** Two critical flows, including one failure path (broker down, duplicate event, OEM format change)

[Write your answer here]

6.5 Algorithms & Data Structures

- **Problem Solved:** e.g. nearest reachable charger, trip segmentation, VIN / DTC parsing
- **Algorithm Used:** Graph, dynamic programming, regex, streaming structures — justify the choice
- **Pseudocode:** For major functions
- **Time & Space Complexity:** Annotate all
- **Scale Tested:** Input size and measured runtime

[Write your answer here]

7. Non-Functional Requirements & Performance Benchmarks

NFR	Target (Case Study)	Achieved	How Measured
Ingest Throughput	100K+ events/sec		
End-to-End Latency	< 2 s dashboard; < 5 s critical alert		
API Latency	p95 < 200 ms; p99 < 500 ms		
Resilience	Recovers after broker / pod failure		
Availability	99.9%, no single point of failure		

- **Load Test Setup:** Tool, data volume, hardware / cluster size
- **Results:** Graphs or screenshots of throughput, p95 / p99 and consumer lag

[Write your answer here]

8. Security & Compliance

- **Threat Model:** STRIDE for the ingestion path and public API; top five threats and controls
- **Authentication & Authorisation:** OAuth2 / OIDC, RBAC, tenant isolation
- **Device Identity & Encryption:** mTLS, TLS in transit, encryption at rest, secrets management
- **Privacy:** How location and driver data is minimised, masked, retained and erased (GDPR, DPDP Act 2023)
- **AI Safety (if applicable):** Prompt-injection defences, tool permissions, human approval, audit log

[Write your answer here]

9. Test Strategy

Test Type	Tools	No. of Tests	Coverage / Result	In CI?
Unit			<i>[% line / branch]</i>	
Integration & Contract				
Acceptance (BDD)			<i>[Scenarios passed]</i>	
Performance / Load / Soak			<i>[Throughput, p95 / p99]</i>	
Security (SAST, DAST, Dependency, Image)			<i>[Critical / high findings]</i>	
Compliance & Chaos				

- **Edge Cases Covered:** e.g. duplicate events, out-of-order data, unknown OEM format, broker outage
- **Sample Input / Output:** If helpful
- **Report Links:** Coverage, load-test and security scan reports

10. Observability

- **Dashboard Screenshot:** Throughput, consumer lag, error rate, latency percentiles
- **Troubleshooting Walk-Through:** How you would find the cause of a latency spike using logs, metrics and traces

[Write your answer here]

11. AI / ML Component (if used)

- **Purpose:** The decision the model or agent supports, and why rules alone were not enough
- **Data & Features:** Training or retrieval data, how it was generated or sourced, leakage checks
- **Model / Agent Design:** Model choice, vector store, tools the agent can call, how access is limited
- **Evaluation:** Metrics against a simple baseline
- **Guardrails & Cost:** Failure handling, hallucination checks, human-in-the-loop, cost and latency per request

[Write your answer here]

12. Architecture Decisions, Risks & Future Enhancements

- **Architecture Decision Records:** 3–5 records: Context → Options Considered → Decision → Consequences (cover databases, messaging and one CAP trade-off)

- **Risks & Technical Debt:** Known limitations and shortcuts taken during the hackathon
- **Future Enhancements:** Next three steps from prototype to pilot

[Write your answer here]

13. Demo Video (5 Minutes Maximum)

Videos longer than 5:00 are cut at 5:00. Show the working product, not only slides. 1080p, clear audio, captions recommended; unlisted YouTube, Vimeo or Drive link.

Time	Segment	What to Show
0:00 – 0:30	Problem	The user, the pain, and one number that proves it matters
0:30 – 1:00	Solution	One-line pitch and key value proposition
1:00 – 3:00	Live Demo	Main user journey end to end on live data, including one real-time event
3:00 – 4:15	Under the Hood	Architecture, load test or dashboard at scale, one failure-recovery moment
4:15 – 5:00	Impact & Next Steps	Measured results, what comes next, and the team

- **Video Link:** *[URL]*

14. Repository Checklist

- **README:** Problem, architecture image, quick start, environment variables, test commands, known issues
- **One-Command Run:** docker compose up (or make run) starts the full stack with the data simulator
- **Structure:** Clear service folders, plus /docs, /infra and /tests
- **CI Pipeline:** Build, lint, all test suites and security scans on every push
- **Hygiene:** No secrets committed, .env.example provided, commits from all members
- **Final Tag:** v1.0-submission

15. Conclusion

- Summary of key learnings
- Strengths of the solution
- Challenges tackled and solved

[Write your answer here]

16. Declarations

- **Open-Source Components:** List with licences (an SBOM file in the repository is acceptable)
- **AI Tools Used:** Coding assistants or LLM APIs, and the parts of the work they were used for
- **Data:** Confirm all data is synthetic or public and contains no real personal data

[Write your answer here]

17. Appendix (if any)

- Additional diagrams or flowcharts
- Full test logs or large datasets used
- References or external sources